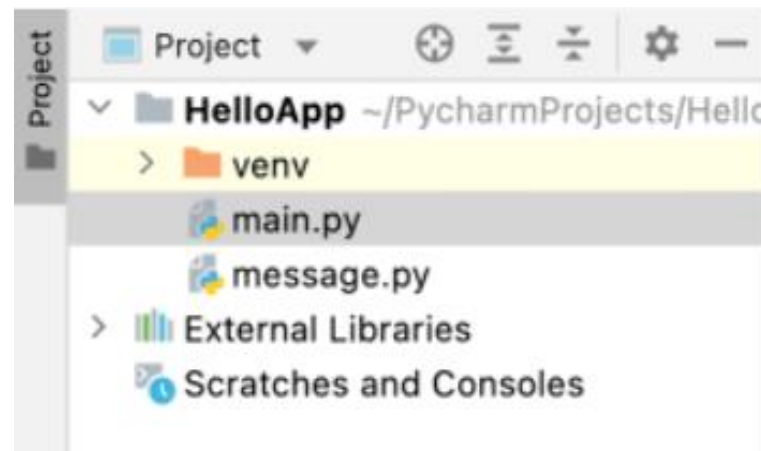


Модули

Дефиниране и свързване на модули

Модулът в Python е файл, който може да се използва повторно и в други програми. За целта се създава файл *.py, който е дефакто името на модула. В него се дефинират една или няколко функции. Ако основният програмен файл се нарича main.py. и се налага да се свържат външни модули към него, тогава първо се дефинира нов модул: създава се нов файл в същата папка като main.py, който в случая е с наименование message.py. Ако се използва PyCharm или друга IDE, и двата файла се поставят в един и същ проект.



Тема 7. Модули. Дефиниране и свързване на модули

В модул **message** имаме следния код:

```
hello = "Hello all"  
def print_message(text):  
    print(f"Message: {text}")
```

Тук са дефинирани променливата **hello** и функцията **print_message**, която приема текст като параметър и го отпечатва в конзолата. Този модул се използва от **main.py**. За да се използва модула, трябва да бъде импортиран с помощта на оператора **import**, последван от името на модула: **import message**. За да получим достъп до функционалността на модула, трябва да получим неговото пространство от имена. По подразбиране то съвпада с името на модула, т.е. в конкретния случай той се нарича **message**. След като се получи пространството от имена на модула, може да се направи обръщение към неговите функции **message.print_message**.

```
import message                                # включване на модул message  
# отпечатване на стойността на променливата hello  
print(message.hello)                           # Hello all  
# извикване на функцията print_message  
message.print_message("Hello work") # Message: Hello work
```

Тема 7. Модули. Дефиниране и свързване на модули

Например, извикване на функция `print_message()` от модула за съобщения:

```
message.print_message("Hello work")
```

И след това може да се стартира основния скрипт `main.py` и той ще използва модула `message.py`. По-конкретно, изходът на конзолата ще бъде:

```
Hello all Message: Hello work
```

Свързване на функционалността на модула към глобалното пространство от имена

Друга опция за персонализиране включва импортиране на функционалност на модула в глобалното пространство от имена на текущия модул с помощта на ключовата дума **from**:

```
from message import print_message
```

```
# извикване на функцията print_message от модула за съобщения
```

```
print_message("Hello work") # Message: Hello work
```

```
# променливата hello от модула за съобщения не е налична, защото не е импортирана
```

```
# print(message.hello)
```

```
# print(hello)
```

В този случай се импортира функцията от модула за съобщения в глобалното пространство от имена `print_message()`. Следователно могат да се използват, без да се указва пространството от имена на модула, все едно, че са дефинирани в същият файл. Всички други функции, променливи от модула не са налични (както е в случая, разгледан по-горе с променливата `hello`).

Тема 7. Модули. Дефиниране и свързване на модули

Ако се налага тяхното използване, тогава те се свързват отделно:

```
from message import print_message
```

```
from message import hello
```

```
# извикване на функцията print_message от модула за съобщения
```

```
print_message("Hello work") # Message: Hello work
```

```
достъп до променливата hello от модула за съобщения
```

```
print(hello) # Hello all
```

Ако се налага импортиране на цялата функционалност в глобалното пространство от имена, вместо да се изброяват имената на отделните функции и променливи, се използва символа звездичка *:

```
from message import * # подобно на * използвана в БД
```

```
# извикване на функцията print_message от модула за съобщения
```

```
print_message("Hello work") # Message: Hello work
```

```
# достъп до променливата hello от модула за съобщения
```

```
print(hello) # Hello all
```

Импортирането в глобалното пространство от имена е **изпълнено с колизии на имена на функции**. Така например: **ако имаме функция, дефинирана в същия файл със същото име, преди да бъде извикана, трябва да се извика функцията на текущия файл:**

Тема 7. Модули. Дефиниране и свързване на модули

```
from message import *  
def print_message(some_text):  
    print(f"Text: {some_text}")  
print_message("Hello work") # Text: Hello work - се прилага функцията от текущия  
файл
```

Задаване на псевдоними

При импортиране на модул и неговата функционалност, може да се зададат псевдоними за тях. За целта се използва ключовата дума **as**, последвана от псевдонима. Например, нека зададем псевдоним за модул:

```
import message as mes  
# съпоставяне на модулът за съобщения с псевдонима mes  
# показване стойността на променливата hello  
print(mes.hello) # Hello all  
# извикване на функцията print_message  
mes.print_message("Hello work") # Message: Hello work
```

В този случай пространството от имена се нарича **mes** и функционалността на модула може да бъде достъпна чрез този псевдоним. По същия начин може да се зададат псевдоними за функционалност на отделните модули:

Тема 7. Модули. Дефиниране и свързване на модули

```
from message import print_message as display
from message import hello as welcome
print(welcome)                # Hello all – променливата hello от модула за съобщения
display("Hello work")         # Message: Hello work - функцията print_message от модул
message
```

В конкретния случай функцията **print_message** от модула за съобщения е настроена на псевдонима **display**, а променливата **hello** е настроена на псевдонима **welcome**. Чрез тези псевдоними се осъществява достъп до тях. Псевдонимите се прилагат:

- За съкращаване на имената на функциите и променливите;
- За придаване на по-описателни имена;
- За избягване на конфликти на имена, когато текущият файл има функционалност със същото име.

Пример:

```
import message as mes # модулет за съобщения се съпоставя с псевдонима mes
# показва стойността на променливата hello
print(mes.hello)      # Hello all
# извикване на функцията print_message
mes.print_message("Hello work") # Message: Hello work
```

Име на модула

В примера по-горе, модулет `main.py`, който е основният, използва модула **`message.py`**. Когато се стартира модула `main.py`, програмата ще свърши цялата необходима работа. Ако обаче се стартира модула **`message.py`** отделно, няма да се види нищо на конзолата. В крайна сметка, модулет за съобщения просто дефинира функция и променлива и не прави нищо друго. Модулет **`message.py`** може да се използва както самостоятелно, така и включен в други модули. Когато се изпълнява модул, средата определя неговото име и го присвоява на глобалната променлива `__name__` (с две долни черти от двете страни). Ако модулет може да се изпълнява, тогава името му е `__main__` (също с две долни черти от всяка страна). Ако модулет се използва в друг модул, тогава в момента на изпълнение името му е същото като името на файла без разширението `.py`. И може да се използва. Нека променим съдържанието на файла **`message.py`**:

```
hello = "Hello all"
def print_message(text):
    print(f"Message: {text}")
def main():
    print_message(hello)
if __name__ == "__main__":
    main()
```

Тема 7. Модули. Дефиниране и свързване на модули

В този случай към модула `message.py` е добавена функция за тестване на функционалността на модула `main`. Може веднага да се стартира файла `message.py` отделно от всички останали и да се тества кода. Обърнете внимание на извикването на основната функция:

```
if __name__ == "__main__":  
    main()
```

Променливата `__name__` сочи към името на модула. За основен модул, който се изпълнява директно, тази променлива винаги ще има стойност `__main__` независимо от името на файла. Следователно, ако се изпълни скрипта `message.py` отделно, самостоятелно, тогава Python ще присвои стойността на променливата `__name__ = __main__`, след което в оператора `if` ще се извика основната функция от същия файл.

Ако обаче изпълним друг скрипт и този - `message.py` - ще бъде включен като допълнителен, променливата `__name__` за `message.py` ще има стойността `message`. И съответно основният метод във файла `message.py` няма да работи. Този подход за проверка на името на модула е по-препоръчителен, отколкото самото извикване на основния метод. Във файла `main.py` може също да се провери дали модулет е основният:

Тема 7. Модули. Дефиниране и свързване на модули

```
import message

def main():
    message.print_message("Hello work") # Message: Hello work

if __name__ == "__main__":
    main()
```

Python предоставя редица вградени модули, които могат да се използват в програмите.

Произволен модул

Модулът `random` контролира генерирането на произволни числа. Основните му функции:

`random()`: генерира произволно число между 0.0 и 1.0

`randint()`: връща произволно число от даден диапазон

`randrange()`: връща произволно число от даден набор от числа

`shuffle()`: разбърква списъка

`choice()`: връща произволен елемент от списъка

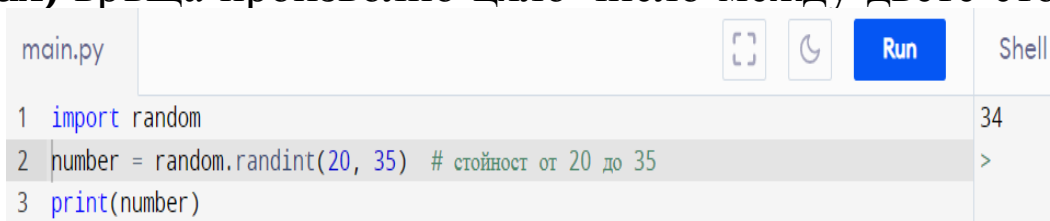
Тема 7. Модули. Дефиниране и свързване на модули

Функцията **random()** връща произволно число с плаваща запетая между 0.0 и 1.0. Ако се налага използването на число от по-голям диапазон, например от 0 до 100, тогава може да се умножи резултата от произволната функция съответно по 100.

```
import random
number = random.random() # стойност от 0.0 до 1.0
print(number)
number = random.random() * 100 # стойност от 0.0 до 100.0
print(number)
```

Функцията **randint(min, max)** връща произволно цяло число между двете стойности min и max.

```
import random
number = random.randint(20, 35)
print(number)
```



```
main.py
1 import random
2 number = random.randint(20, 35) # стойност от 20 до 35
3 print(number)
```

34

Функцията **randrange()** връща произволно цяло число от даден набор от числа. Има три форми:

randrange(stop): като набор от числа, от които се извлича произволната стойност, ще се използва диапазонът от 0 до точката избрана за stop.

randrange(start, stop): наборът от числа представлява диапазон от начална до крайна точка

randrange(start, stop, step): набор от числа представлява диапазон от начална до крайна точка, като всяко число в диапазона се различава от предишното със стъпка

Тема 7. Модули. Дефиниране и свързване на модули

main.py



Run

Shell

```
1 import random
2 number = random.randrange(10) # стойност от 0 до 10
3 print(number)
4 number = random.randrange(2, 10) # стойност в диапазона 2, 3, 4, 5, 6, 7, 8, 9
5 print(number)
6 number = random.randrange(2, 10, 2) # стойност в диапазона 2, 4, 6, 8
7 print(number)
```

3
7
2
>

Работа със списък

За да работите със списъци, в произволния модул са дефинирани две функции: функцията `shuffle()` разбърква списъка произволно, а функцията `choice()` връща един произволен елемент от списъка:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
```

```
random.shuffle(numbers)
```

```
print(numbers)
```

```
random_number =
```

```
random.choice(numbers)
```

```
print(random_number)
```

main.py



Run

Shell

```
1 import random
2 numbers = [1, 2, 3, 4, 5, 6, 7, 8]
3 random.shuffle(numbers)
4 print(numbers)
5 random_number = random.choice(numbers)
6 print(random_number)
```

[3, 4, 7, 1, 5, 6, 8, 2]
6
> |

Математически модул

Вграденият математически модул в Python предоставя набор от функции за извършване на математически, тригонометрични и логаритмични операции. Някои от основните характеристики на модула:

- **pow(num, power):** повдигане на num на степен power
- **sqrt(num):** корен квадратен от num
- **ceil(num):** закръгляне на числото до най-близкото цяло число
- **floor (num):** закръгляне на числото до най-близкото, най-малко цяло число
- **factorial(num):** факториел на числото
- **degrees (rad):** преобразуване от радиани в градуси
- **radians(grad):** преобразуване от градуси в радиани
- **cos(rad):** косинус на ъгъла в радиани
- **sin(rad):** синус на ъгъла в радиани
- **tan(rad):** тангенс на ъгъла в радиани
- **acos(rad):** аркус синус на ъгъла в радиани
- **asin(rad):** аркус синус на ъгъла в радиани
- **atan(rad):** аркус тангенс на ъгъла в радиани
- **log(n, base):** основен логаритъм на n
- **log10(n):** логаритъм на база 10 от n

Тема 7. Модули. Дефиниране и свързване на модули

Пример за използване на някои функции:

```
import math
```

```
# повдигане на числото 2 на степен 3
```

```
n1 = math.pow(2, 3)
```

```
print(n1) # 8
```

```
# същата операция може да се направи по този начин
```

```
n2 = 2**3
```

```
print(n2)
```

```
# квадратен корен на число
```

```
print(math.sqrt(9)) # 3
```

```
# най-близкото цяло число
```

```
print(math.ceil(4.56)) # 5
```

```
# закръгляне на числото до най-близкото, най-малко цяло число
```

```
print(math.floor(4.56)) # 4
```

```
# преобразуване на радиан в градус
```

```
print(math.degrees(3.14159)) # 180
```

```
# преобразуване на градуси в радиани
```

```
print(math.radians(180)) # 3.1415.....
```

```
# косинус
```

```
print(math.cos(math.radians(60))) # 0.5
```

```
# синус
```

```
print(math.sin(math.radians(90))) # 1.0
```

```
# тангенс
```

```
print(math.tan(math.radians(0))) # 0.0
```

```
print(math.log(8,2)) # 3.0
```

```
print(math.log10(100)) # 2.0
```

```
mcin.py  [Run] Shell
1 import math
2 # повдигане на числото 2 на степен 3
3 n1 = math.pow(2, 3)
4 print(n1) # 8
5 # същата операция може да се направи по този начин
6 n2 = 2**3
7 print(n2)
8 # квадратен корен на число
9 print(math.sqrt(9)) # 3
10 # най-близкото цяло число
11 print(math.ceil(4.56)) # 5
12 # най-близкото най-голямо цяло число
13 print(math.floor(4.56)) # 4
14 # преобразуване на радиан в градус
15 print(math.degrees(3.14159)) # 180
16 # преобразуване на градуси в радиани
17 print(math.radians(180)) # 3.1415.....
18 # косинус
19 print(math.cos(math.radians(60))) # 0.5
20 # Синус
21 print(math.sin(math.radians(90))) # 1.0
22 # тангенс
23 print(math.tan(math.radians(0))) # 0.0
24
25 print(math.log(8,2)) # 3.0
26 print(math.log10(100)) # 2.0
```

8.0
8
3.0
5
4
179.9998479605043
3.141592653589793
0.5000000000000001
1.0
0.0
3.0
2.0
>

Тема 7. Модули. Дефиниране и свързване на модули

Математическият модул също така предоставя редица вградени константи, като **pi** и **e**:

```
import math
radius = 30
# лице на кръг с радиус 30
area = math.pi * math.pow(radius, 2)
print(area)
# естествен логаритъм от 10
number = math.log(10, math.e)
print(number)
```



The screenshot shows a code editor with a file named 'main.py'. The code in the editor is as follows:

```
1 import math
2 radius = 30
3 # лице на кръг с радиус 30
4 area = math.pi * math.pow(radius, 2)
5 print(area)
6 # естествен логаритъм от 10
7 number = math.log(10, math.e)
8 print(number)
```

To the right of the code editor is a 'Shell' window showing the output of the script:

```
2827.4333882308138
2.302585092994046
> |
```

Локален модул

При форматирането на числа Python използва **англосаксонската система по подразбиране**, при която цифрите на цяло число са разделени една от друга със запетай, а дробната част от цялото число е разделена с точка. В континентална Европа, например, се използва различна система, при която цифрите са разделени с точка, а дробната и цялата част са разделени със запетая:

англосаксонска система

1,234.567

европейска система

1.234,567

Тема 7. Модули. Дефиниране и свързване на модули

За да реши проблема с форматирането за конкретна култура, Python има вграден локален модул.

Функцията **setlocale()** е дефинирана в локалния модул, за да зададе локалната култура. Необходими са два параметъра:

setlocale(category, locale)

Първият параметър определя категорията, към която се прилага функцията - числа, валути или и двете числа и валути. **Като стойност за параметър** може да **се предаде една от следните константи**:

- **LC_ALL**: прилага локализация към всички категории - форматиране на числа, валути, дати и т.н.
- **LC_NUMERIC**: прилага локализация към числата
- **LC_MONETARY**: прилага локализация към валути
- **LC_TIME**: прилага локализация към дати и часове
- **LC_CTYPE**: прилага локализация при преобразуване на знаци в главни или малки букви
- **LC_COLLATE**: прилага локализация при сравняване на низове

Тема 7. Модули. Дефиниране и свързване на модули

Вторият параметър на функцията **setlocale** определя локалната култура, която да се използва. В ОС Windows се използва ISO кода на държавата от два знака, например за САЩ - "us", за Германия - "de", за Русия - "ru". Но в MacOS трябва да посочите кода на езика и кода на държавата, например за английски в САЩ - "en_US", за немски в Германия - "de_DE", за руски в Русия - "ru_RU". По подразбиране всъщност културата е "en_US".

Директно за форматиране на числа и валути, локалният модул предоставя две функции:

- **currency(num)**: форматира валута
- **format_string(str, num)**: замества числото num вместо заместителя в низа str

Прилагат се следните заместители:

- о **d**: за цели числа
- о **f**: за числа с плаваща запетая
- о **e**: за експоненциална нотация

Всеки заместител е предшестван от знак за процент %, например:

"%d"

Когато се извеждат дробни числа преди заместителя, след точката, може да се посочи броят на знаците в дробната част, които трябва да се показват:

%.2f # два знака в дробната част

Тема 7. Модули. Дефиниране и свързване на модули

Прилагане на локализация на числа и валути в немската култура:

```
import locale
```

```
locale.setlocale(locale.LC_ALL, "de")    # за Windows
```

```
# locale.setlocale(locale.LC_ALL, "de_DE") # за MacOS
```

```
number = 12345.6789
```

```
formatted = locale.format_string("%f", number)
```

```
print(formatted)    # 12345,678900
```

```
formatted = locale.format_string("%.2f", number)
```

```
print(formatted)    # 12345,68
```

```
formatted = locale.format_string("%d", number)
```

```
print(formatted)    # 12345
```

```
formatted = locale.format_string("%e", number)
```

```
print(formatted)    # 1,234568e+04
```

```
money = 234.678
```

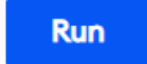
```
formatted = locale.currency(money)
```

```
print(formatted)    # 234,68 €
```

main.py	Run	Shell
1 import locale		12345.678900
2		12345.68
3 number = 12345.6789		12345
4 formatted = locale.format_string("%f", number)		1.234568e+04
5 print(formatted) # 12345,678900		>
6		
7 formatted = locale.format_string("%.2f", number)		
8 print(formatted) # 12345,68		
9		
10 formatted = locale.format_string("%d", number)		
11 print(formatted) # 12345		
12		
13 formatted = locale.format_string("%e", number)		
14 print(formatted) # 1,234568e+04		
--		

Тема 7. Модули. Дефиниране и свързване на модули

Ако като втори параметър се подаде празен низ вместо конкретен код, тогава Python ще използва културата, която се прилага на текущата работеща машина. И с помощта на функцията `getlocale()` може да получите тази култура:

main.py	  	Shell
<pre>1 import locale 2 3 locale.setlocale(locale.LC_ALL, "") 4 5 number = 12345.6789 6 formatted = locale.format_string("%.02f", number) 7 print(formatted) # 12345,68 8 print(locale.getlocale()) 9 # ('Russian_Russia', '1251') - Windows 10 # ('ru_RU', 'UTF-8') - MacOS</pre>		<pre>12345.68 ('en_US', 'UTF-8') > </pre>

В зависимост от системата изходът може да се различава.

Десетичен модул

Когато се работи с числа с плаваща запетая (т.е. `float`), се получава така, че в резултат на изчисленията се получават не съвсем правилен резултат:

```
number = 0.1 + 0.1 + 0.1
print(number)      # 0.30000000000000004
```

Тема 7. Модули. Дефиниране и свързване на модули

Проблемът може да бъде решен с помощта на функцията `round()`, която ще закръгли числото. Друг възможен начин, който може да се използва е вградения десетичен модул.

Ключовият компонент за работа с числа в този модул е класът `Decimal`. За да се използва, трябва да се създаде неговият обект с помощта на конструктора. На конструктора се предава стойност на низ, която представлява число:

```
from decimal import Decimal  
number = Decimal("0.1")
```

След това обектът `Decimal` може да се използва в аритметични операции:

```
from decimal import Decimal  
number = Decimal("0.1")  
number = number + number + number  
print(number)    # 0.3
```

Може да се използват цели числа в десетични операции:

```
number = Decimal("0.1")  
number = number + 2
```

Въпреки това, не може да се смесва `float` и десетични дробни числа в операции:

```
number = Decimal("0.1")  
number = number + 0.1  # тук възниква грешка
```

Тема 7. Модули. Дефиниране и свързване на модули

main.py	  	Shell	
<pre>1 from decimal import Decimal 2 number = Decimal("0.1") 3 number = number + number + number 4 print(number) # 0.3 5 6 number = Decimal("0.1") 7 number = number + 2 8 print(number) # 2.1 9 10 number = Decimal("0.10") 11 number = 3 * number 12 print(number) # 0.30 13 14 number = Decimal("0.1") 15 number = number + 0.1 # тук възниква грешка 16 print(number)</pre>		<pre>0.3 2.1 0.30 Traceback (most recent call last): File "<string>", line 15, in <module> TypeError: unsupported operand type(s) for +: 'decimal.Decimal' and 'float' > </pre>	

Тема 7. Модули. Дефиниране и свързване на модули

С помощта на допълнителни знаци може да се определи колко знака ще има в дробната част на числото:

```
number = Decimal("0.10")
number = 3 * number
print(number)    # 0.30
```

Низът "0.10" определя два знака след десетичната запетая, дори ако последните знаци представляват нула. Съответно "0,100" представлява три знака след десетичната запетая.

Закръгляне на числата

Десетичните обекти имат метод `quantize()`, който позволява закръгляне на числата. Обектът `Decimal` също се предава на този метод като първи аргумент, който определя формата на закръгляне на числото:

```
from decimal import Decimal
```

```
number = Decimal("0.444")
number = number.quantize(Decimal("1.00"))
print(number)    # 0.44
```

```
number = Decimal("0.555678")
print(number.quantize(Decimal("1.00")))    # 0.56
```

Тема 7. Модули. Дефиниране и свързване на модули

```
number = Decimal("0.999")  
print(number.quantize(Decimal("1.00")))    # 1.00
```

main.py	Run	Shell
1 from decimal import Decimal		0.44
2		0.56
3 number = Decimal("0.444")		1.00
4 number = number.quantize(Decimal("1.00"))		>
5 print(number) # 0.44		
6		
7 number = Decimal("0.555678")		
8 print(number.quantize(Decimal("1.00"))) # 0.56		
9 number = Decimal("0.999")		
10 print(number.quantize(Decimal("1.00"))) # 1.00		

Използваният низ "1.00" показва, че закръглението ще се направи до два знака след десетичната запетая.

По подразбиране закръглението се описва с константата ROUND_HALF_EVEN, която се закръгля до най-близкото четно число, ако частта, която трябва да се закръгли, е 5.

Пример:

```
from decimal import Decimal, ROUND_HALF_EVEN  
number = Decimal("10.025")    # 2 – най-близкото четно число  
print(number.quantize(Decimal("1.00"), ROUND_HALF_EVEN))    # 10.02  
number = Decimal("10.035")    # 4 – най-близкото четно число  
print(number.quantize(Decimal("1.00"), ROUND_HALF_EVEN))    # 10.04
```

Тема 7. Модули. Дефиниране и свързване на модули

Стратегията за закръгление се предава като втори параметър за квантуване. Низът "1.00" означава, че закръглението ще премине до две числа в дробната част. Но в първия случай "10,025" - вторият знак е 2 - е четно число, следователно, въпреки факта, че следващото число е 5, двете не се закръглят до три. Във втория случай "10.035" - вторият знак е 3 - е нечетно число, най-близкото четно число е 4, така че 35 се закръгля до 40. Това поведение на закръгление може да не е желателно за всеки, в който случай може да бъде отменено с помощта на една от следните константи:

- **ROUND_HALF_UP:** Закръгля число, ако е последвано от 5 или по-високо число
- **ROUND_HALF_DOWN:** Закръгля число нагоре, ако е последвано от число, по-голямо от 5

```
number = Decimal("10.026")
print(number.quantize(Decimal("1.00"), ROUND_HALF_DOWN))    # 10.03
number = Decimal("10.025")
print(number.quantize(Decimal("1.00"), ROUND_HALF_DOWN))    # 10.02
```

- **ROUND_05UP:** Закръгля 0 надолу до едно, ако е последвано от 5 или повече
- ```
number = Decimal("10.005")
print(number.quantize(Decimal("1.00"), ROUND_05UP)) # 10.01
number = Decimal("10.025")
print(number.quantize(Decimal("1.00"), ROUND_05UP)) # 10.02
```

## Тема 7. Модули. Дефиниране и свързване на модули

- **ROUND\_CEILING:** Закръгля число нагоре, независимо кое число идва след него

```
number = Decimal("10.021")
print(number.quantize(Decimal("1.00"), ROUND_CEILING)) # 10.03
```

```
number = Decimal("10.025")
print(number.quantize(Decimal("1.00"), ROUND_CEILING)) # 10.03
```

- **ROUND\_FLOOR:** Не закръгля числото, без значение кое число идва след него

```
number = Decimal("10.021")
print(number.quantize(Decimal("1.00"), ROUND_FLOOR)) # 10.02
```

```
number = Decimal("10.025")
print(number.quantize(Decimal("1.00"), ROUND_FLOOR)) # 10.02
```

## Тема 7. Модули. Дефиниране и свързване на модули

main.py



Run

Shell

```
1 from decimal import Decimal, ROUND_HALF_EVEN
2 number = Decimal("10.025") # 2 - най-близкото четно число
3 print(number.quantize(Decimal("1.00"), ROUND_HALF_EVEN)) # 10.02
4 number = Decimal("10.035") # 4 - най-близкото четно число
5 print(number.quantize(Decimal("1.00"), ROUND_HALF_EVEN)) # 10.04
6 from decimal import Decimal, ROUND_HALF_DOWN
7 number = Decimal("10.026")
8 print(number.quantize(Decimal("1.00"), ROUND_HALF_DOWN)) # 10.03
9 number = Decimal("10.025")
10 print(number.quantize(Decimal("1.00"), ROUND_HALF_DOWN)) # 10.02
11 from decimal import Decimal, ROUND_05UP
12 number = Decimal("10.005")
13 print(number.quantize(Decimal("1.00"), ROUND_05UP)) # 10.01
14 number = Decimal("10.025")
15 print(number.quantize(Decimal("1.00"), ROUND_05UP)) # 10.02
16 from decimal import Decimal, ROUND_CEILING
17 number = Decimal("10.021")
18 print(number.quantize(Decimal("1.00"), ROUND_CEILING)) # 10.03
19 number = Decimal("10.025")
20 print(number.quantize(Decimal("1.00"), ROUND_CEILING)) # 10.03
21 from decimal import Decimal, ROUND_FLOOR
22 number = Decimal("10.021")
23 print(number.quantize(Decimal("1.00"), ROUND_FLOOR)) # 10.02
24 number = Decimal("10.025")
25 print(number.quantize(Decimal("1.00"), ROUND_FLOOR)) # 10.02
```

## **Използвана литература:**

- [1]. Joakim Sundnes, Introduction to Scientific Programming with Python, Simula Springer Briefs on Computing, 2020
- [2]. Brian Heinold, A Practical Introduction to Python Programming, Department of Mathematics and Computer Science Mount St. Mary's University, 2012
- [3]. David Mertz, Functional Programming in Python, O'Reilly Media, 2015
- [4]. Steven Lott, Functional Python Programming, Published by Packt Publishing Ltd., 2015
- [5]. Mark Lutz, Learning Python, Fourth Edition, O'Reilly Media, 2009